

Castilfac

Documentacion Tecnica del Proyecto
SaaS de Gestion para Carpinteria Metalica y Cristaleria

Repositorio: github.com/PabloCstlpz/castilfac

Autor: Pablo M. Del Castillo Lopez

Mayo 2026

Indice

1. Resumen del Proyecto	3
2. Arquitectura General	4
3. Tecnologias del Stack	5
4. Backend: Node.js + Express	6
5. Frontend: Angular 21 + Tailwind	9
6. Base de Datos: MariaDB + Sequelize	12
7. Modelo de Datos Relacional	13
8. API REST - Endpoints	15
9. Seguridad	17
10. Multi-Tenencia (SaaS)	18
11. Stripe y Suscripciones	19
12. Internacionalizacion (i18n)	20
13. Docker y Despliegue	21
14. Estructura del Repositorio	22
15. Ejecucion Local	23
16. Mejoras Potenciales	24

1. Resumen del Proyecto

Castilfac es un SaaS (Software as a Service) full stack para la gestion integral de talleres de carpinteria metalica, aluminio, PVC y cristaleria. Permite a multiples empresas (clientes del SaaS) gestionar su catalogo de productos y materiales, crear presupuestos con PDF, administrar pedidos, controlar clientes, asignar operarios, y gestionar finanzas, todo con precios personalizados por empresa y un sistema de suscripciones mediante Stripe.

El proyecto fue desarrollado como proyecto final del ciclo de Desarrollo de Aplicaciones Web (DAW) y esta desplegado en un servidor real mediante Docker y Portainer, con acceso publico a traves de Cloudflare Tunnel.

Ficha del Proyecto

Repositorio: github.com/Pablocstlpz/castilfac (publico)

Estado: Funcional en produccion (Docker + Portainer)

Stack Frontend: Angular 21 + Tailwind CSS + Angular Material

Stack Backend: Node.js + Express 5 + Sequelize ORM

Base de datos: MariaDB 10.11 (MySQL compatible)

Pagos: Stripe (suscripciones mensuales/anuales)

Auth: JWT + Google OAuth + bcrypt

Lenguajes: TypeScript, JavaScript, HTML, CSS, SQL

Total archivos: ~390

Despliegue: Docker Compose (3 servicios: web, api, db)

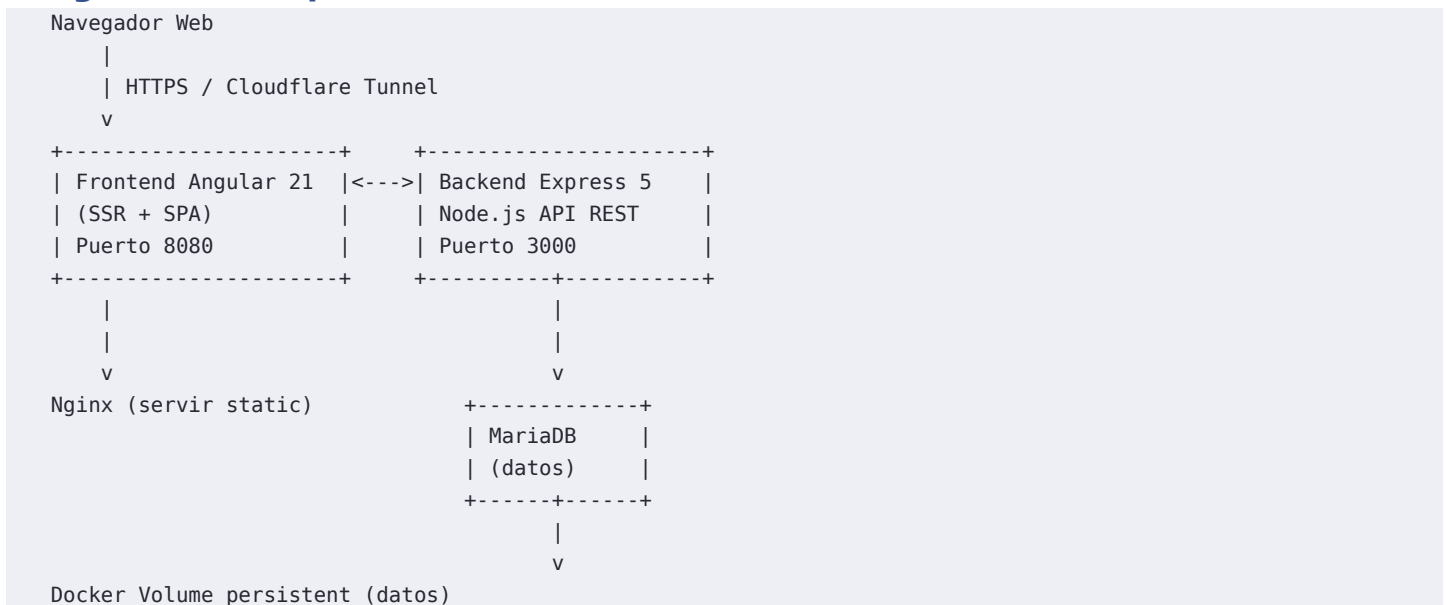
2. Arquitectura General

Castilfac sigue una arquitectura de tres capas: Frontend Angular, Backend Node.js/Express y Base de Datos MariaDB. Todo orquestado con Docker Compose y desplegado en un servidor real con Portainer y Cloudflare Tunnel.

Caracteristicas arquitectonicas clave:

- Multi-tenencia: cada empresa (inquilino) tiene sus datos aislados por empresa_id
- Arquitectura SaaS: un unico despliegue sirve a multiples empresas
- Comunicacion HTTP/JSON con JWT en todas las peticiones autenticadas
- Separacion frontend/backend en contenedores Docker independientes
- Base de datos MariaDB compartida con aislamiento logico por empresa
- Servicio de correo SMTP para notificaciones y verificaciones

Diagrama de Arquitectura



Principios de Diseno

- Arquitectura SaaS multi-inquilino con aislamiento por empresa_id
- API REST versionada, con rutas por dominio funcional
- Validacion de entrada con express-validator
- Autenticacion JWT + Google OAuth
- Rate limiting por IP (express-rate-limit)
- Multi-idioma: i18n con ngx-translate (es/en)
- Generacion de PDFs en cliente con jspdf
- Despliegue Docker automatizado con Portainer

3. Tecnologias del Stack

Frontend

Framework: Angular 21 (standalone components)

Lenguaje: TypeScript

Estilos: Tailwind CSS 4 + Angular Material 21 + Font Awesome 7

SSR: Angular Universal (server.ts)

I18n: @ngx-translate/core + @ngx-translate/http-loader

PDF: jspdf 4 + jspdf-autotable

Alertas: SweetAlert2 11

Reactive: RxJS ~7.8

SSR Server: Express (integrado en Angular)

Backend

Runtime: Node.js (ES Modules)

Framework: Express 5

ORM: Sequelize 6 (MySQL/MariaDB)

Driver BD: mysql2

Autenticacion: jsonwebtoken + bcrypt + google-auth-library

Pagos: stripe SDK 22

Email: nodemailer

Seguridad: helmet + cors + express-rate-limit

Validacion: express-validator

Base de Datos

Motor: MariaDB 10.11

ORM: Sequelize 6

Modelo: Relacional (14 tablas)

Migraciones: Automaticas via sync() en desarrollo

DevOps / Infraestructura

Contenedores: Docker + Docker Compose

Gestion: Portainer CE

Proxy/Tunel: Cloudflare Tunnel

Servidor: Ubuntu Server, IP: 79.72.48.126

Dominio: castilfac.pablosrv.com

4. Backend: Node.js + Express

4.1 Estructura del Backend

```

backend/
|-- Dockerfile                # Imagen Node.js
|-- package.json              # Dependencias (ESM)
|-- src/
|  |-- app.js                 # Entry point + rutas
|  |-- config.js              # Variables de entorno
|  |-- mailer.js              # Servicio SMTP
|  |-- data/
|     |-- db.js               # Conexion Sequelize + sync
|-- controllers/              # Logica de negocio
|  |-- auth.controller.js
|  |-- usuarios.controller.js
|  |-- empresas.controller.js
|  |-- clientes.controller.js
|  |-- materiales.controller.js
|  |-- categorias.controller.js
|  |-- elementos.controller.js
|  |-- elementosMateriales.controller.js
|  |-- preciosEmpresa.controller.js
|  |-- presupuestos.controller.js
|  |-- pedidos.controller.js
|  |-- plantillasProducto.controller.js
|  |-- plantillasMateriales.controller.js
|  |-- historialPreciosBase.controller.js
|  |-- historialPreciosEmpresa.controller.js
|  |-- suscripcion.controller.js
|  |-- stripe.controller.js
|-- models/                   # Modelos Sequelize
|  |-- 14 modelos + associations.js
|-- routes/                   # Definicion de rutas
|  |-- 16 archivos de rutas
|-- middlewares/              # Middleware personalizado
|  |-- auth.middleware.js     # Verificacion JWT
|  |-- checkSuscripcion.middleware.js
|  |-- rateLimit.middleware.js
|-- utils/                    # Utilidades
|  |-- tenant.js              # Contexto multi-inquilino
|  |-- seguridad.js           # Hashing, JWT
|  |-- logger.js              # Logging
|  |-- regex.js               # Expresiones regulares
|-- validators/               # Validacion de entrada
|  |-- cliente.validator.js
|  |-- empresa.validator.js
|  |-- usuario.validator.js
|-- _handle.js

```

4.2 Punto de Entrada (app.js)

El archivo app.js configura Express con todos los middleware necesarios: helmet para seguridad de cabeceras HTTP, CORS con lista blanca de origenes, rate limiting para proteccion contra abusos, y parseo de JSON. El webhook de Stripe se registra antes de express.json() para recibir el body en crudo y poder verificar la firma.

Todas las rutas se montan bajo /api/ con una organizacion por dominio funcional. Las asociaciones de Sequelize se cargan por efecto secundario del import de associations.js para que esten disponibles globalmente.

4.3 Configuracion y .env

El backend se configura mediante variables de entorno. En produccion estas se inyectan desde las environment variables del stack de Portainer. En desarrollo se usa un archivo .env.

- PORT: puerto del servidor (3000)
- DB_HOST / DB_USER / DB_PASSWORD / DB_DATABASE: conexion MariaDB
- SECRET_KEY / REFRESH_SECRET_KEY: claves JWT
- URL / FRONTEND_URL: URLs publicas
- STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET: claves de pago
- GOOGLE_CLIENT_ID: OAuth de Google
- EMAIL_USER / EMAIL_PASS: credenciales SMTP

4.4 Middleware de Autenticacion

auth.middleware.js verifica el token JWT en el header Authorization, extrae el payload (usuario_id, empresa_id, rol) y lo inyecta en req.usuario. Se usa junto con checkSuscripcion.middleware.js que valida que la empresa tenga una suscripcion activa antes de permitir operaciones.

El middleware rateLimit.middleware.js configura limitadores por IP con express-rate-limit para proteger endpoints sensibles como login.

4.5 Servicio de Email

mailer.js implementa el envio de correos SMTP con nodemailer. Se utiliza para notificaciones internas del sistema, como alertas de actividad y comunicaciones con los administradores.

5. Frontend: Angular 21 + Tailwind

5.1 Estructura del Frontend

```
frontend/
|-- Dockerfile, nginx.conf, angular.json
|-- tailwind.config.js
|-- src/
|   |-- index.html, main.ts, server.ts (SSR)
|   |-- styles.css, material-theme.scss
|   |-- app/
|       |-- app.ts/html/css, app.routes.ts
|       |-- components/
|           |-- enlaces-generales/      # Paginas publicas
|           |   |-- welcome, company, team
|           |   |-- contacto, feature, planes
|           |   |-- sobrenosotros, politicas, terminos
|           |-- login-iniciarsesion/    # Login + estados
|           |-- registro-creacion/      # Registro + verif.
|           |-- inicioadmin/           # Panel admin
|           |   |-- admin-layout, barra-lateral
|           |   |-- clientes/          # CRUD completo
|           |   |-- catalogo-y-precios/ # Materiales
|           |   |-- presupuestos/      # Presupuestos+PDF
|           |   |-- pedidos/           # Gestion pedidos
|           |   |-- finanzas/          # Dashboard finanzas
|           |   |-- gestion-personal/   # Usuarios empresa
|           |-- iniciooperario/         # Panel operario
|           |-- iniciosuperadmin/       # Panel superadmin
|           |-- restablecer-password/
|           |-- stripe-pagos/           # Checkout Stripe
|           |-- verificacion-perfil/
|           |-- nopermitir/             # 403, 404, etc.
|       |-- guards/                     # auth, role, subscription
|       |-- interceptors/               # Auth interceptor
|       |-- interfaces/                 # 14 interfaces TS
|       |-- services/                   # 19 servicios HTTP
|       |-- shared/                     # regex
```

5.2 Roles y Permisos

El sistema define tres roles de usuario con permisos progresivos:

- superadmin: acceso global a todas las empresas y datos del sistema
- admin: gestion completa de su empresa (clientes, presupuestos, catalogo, personal)
- operario: acceso limitado a pedidos asignados y tareas de taller

Los guards `role.guard.ts` y `subscription.guard.ts` protegen las rutas segun el rol del usuario y el estado de la suscripcion de la empresa.

5.3 Sistema de Rutas

El sistema de routing define aproximadamente 40 rutas con proteccion escalonada: `AuthGuard` (sesion activa), `RoleGuard` (rol minimo requerido), `SubscriptionGuard` (suscripcion activa). Incluye rutas publicas (welcome, planes, contacto), rutas de autenticacion (login, registro, verificacion), y rutas privadas segregadas por panel (admin, operario, superadmin).

5.4 Interceptor HTTP

`AuthInterceptor` anade automaticamente el token JWT en todas las peticiones HTTP, asi como el header

X-Empresa-Id para el contexto multi-inquilino. Maneja errores 401/403 limpiando la sesion y redirigiendo al login.

5.5 Servicios HTTP

19 servicios Angular encapsulan todas las llamadas HTTP. Destacan un `base-http.service.ts` que abstrae la logica comun de peticiones CRUD con manejo de errores y tipado generico, y un servicio `pdf.ts` que genera presupuestos en PDF usando `jspdf` + `jspdf-autotable`.

5.6 Internacionalizacion

La aplicacion soporta espanol e ingles mediante `ngx-translate`. Los archivos de traduccion (`es.json` con ~50KB, `en.json`) contienen todas las cadenas de la interfaz. La deteccion de idioma se hace automaticamente o por preferencia del usuario.

6. Base de Datos: MariaDB + Sequelize

La base de datos utiliza MariaDB 10.11 con Sequelize 6 como ORM. El modelo relacional tiene 14 tablas principales, con la empresa_id como clave de particion multi-inquilino en la mayoria de tablas.

6.1 Conexion y Sincronizacion (db.js)

db.js establece la conexion con MariaDB mediante Sequelize, leyendo las credenciales de variables de entorno. En desarrollo, sync() crea/actualiza las tablas automaticamente. En produccion se recomienda usar migraciones.

7. Modelo de Datos Relacional

Tablas Principales

empresas

id, nombre, nif, direccion, telefono, email,
plan, stripe_subscription_id, estado_suscripcion

usuarios

id, empresa_id (FK), nombre, email, password (bcrypt),
rol (superadmin|admin|operario), activo, ultimo_acceso

clientes

id, empresa_id (FK), nombre, nif, direccion, telefono,
email, notas

presupuestos

id, empresa_id (FK), cliente_id (FK), usuario_id (FK),
fecha, estado, total, pdf_url, notas, elementos[]

pedidos

id, empresa_id (FK), cliente_id (FK),
operario_asignado_id (FK), fecha, estado,
fecha_entrega, total, notas

materiales

id, empresa_id (FK), categoria_id (FK), nombre,
precio_base, unidad, stock

categorias

id, empresa_id (FK), nombre, descripcion

elementos (lineas de presupuesto)

id, presupuesto_id (FK), descripcion, cantidad,
precio_unitario, total

Otras tablas

- elemento_material: relacion N:M entre elementos y materiales
- precios_empresa: precios personalizados por empresa
- historial_precios_base: historial de cambios en precio_base
- historial_precios_empresa: historial de cambios en precios por empresa
- plantillas_producto: plantillas de productos para presupuestos rapidos
- plantillas_material: materiales asociados a cada plantilla

Relaciones Clave (associations.js)

Todas las relaciones se centralizan en associations.js. La empresa es la entidad raiz: Empresa tiene muchos Usuarios, Clientes, Pedidos, Presupuestos, Materiales, PreciosEmpresa, etc. Cliente tiene Pedidos y Presupuestos. Usuario tiene Pedidos (como operario asignado) y Presupuestos (como autor).

8. API REST - Endpoints

La API se monta bajo /api/ y se organiza por dominio funcional (~60 endpoints). Los endpoints marcados con [Auth] requieren JWT. Los que llevan [Sub] ademas requieren suscripcion activa.

Autenticacion

POST	/api/login	# Login con email/password
POST	/api/login/google	# Login con Google OAuth
POST	/api/refresh-token	# Renovar JWT
POST	/api/logout	# Cerrar sesion

Empresas

POST	/api/empresas	# [Admin] Crear empresa
GET	/api/empresas	# [Admin] Listar empresas
GET	/api/empresas/:id	# [Admin] Datos empresa
PUT	/api/empresas/:id	# [Admin] Actualizar
DELETE	/api/empresas/:id	# [SuperAdmin] Eliminar

Usuarios

GET	/api/usuarios	# [Auth] Listar usuarios
POST	/api/usuarios	# [Auth] Crear usuario
GET	/api/usuarios/:id	# [Auth] Datos usuario
PUT	/api/usuarios/:id	# [Auth] Actualizar
DELETE	/api/usuarios/:id	# [Auth] Eliminar
GET	/api/usuarios/me	# [Auth] Mi perfil

Clientes / Materiales / Categorías

CRUD completo para cada recurso:
 GET, POST, PUT, DELETE /api/clientes
 GET, POST, PUT, DELETE /api/materiales
 GET, POST, PUT, DELETE /api/categorias

Presupuestos

GET	/api/presupuestos	# [Auth] Listar
POST	/api/presupuestos	# [Auth] Crear
GET	/api/presupuestos/:id	# [Auth] Detalle
PUT	/api/presupuestos/:id	# [Auth] Actualizar
DELETE	/api/presupuestos/:id	# [Auth] Eliminar

Pedidos

GET	/api/pedidos	# [Auth] Listar
POST	/api/pedidos	# [Auth] Crear
GET	/api/pedidos/:id	# [Auth] Detalle
PUT	/api/pedidos/:id	# [Auth] Actualizar
DELETE	/api/pedidos/:id	# [Auth] Eliminar

Stripe (Pagos)

POST	/api/stripe/checkout	# [Auth] Crear sesion pago
POST	/api/stripe/webhook	# Webhook Stripe (body raw)
GET	/api/stripe/portal	# [Auth] Portal de Stripe

Suscripcion

GET	/api/suscripcion/estado	# [Auth] Estado suscripcion
POST	/api/suscripcion/cancelar	# [Auth] Cancelar suscripcion

Historial de Precios

GET	/api/historial-precios-base	# [Auth] Historial cambios
GET	/api/historial-precios-empresa	# [Auth] Historial por empresa

9. Seguridad

Autenticacion

- Contraseñas hasheadas con bcrypt (cost factor 10+)
- JWT dual: access token (corta duración) + refresh token
- Google OAuth como alternativa de login
- Verificación de token en cada petición protegida

Proteccion del Servidor

- Helmet: cabeceras de seguridad HTTP (X-Frame-Options, HSTS, etc.)
- CORS: lista blanca de orígenes permitidos (incluye localhost y FRONTEND_URL)
- Rate limiting: express-rate-limit por IP en login y endpoints sensibles
- Trust proxy: configurado para que funcione detrás de reverse proxy/Cloudflare
- Stripe webhook: verificación de firma criptográfica en el body raw

Multi-Tenencia

- Aislamiento de datos por empresa_id en todas las queries
- Middleware de tenant: inyecta empresa_id desde el JWT en cada request
- Un usuario solo ve/modifica datos de su propia empresa
- Superadmin tiene visibilidad global

Validacion de Datos

- express-validator en los endpoints críticos
- Validación de email, NIF, teléfono con regex desde utils/regex.js
- Validación también en frontend antes de enviar al servidor

Auditoria

El proyecto incluye auditoria.md y solucion-auditoria.md que documentan un proceso de auditoria de seguridad realizado sobre la aplicacion, incluyendo hallazgos y las soluciones implementadas para cada vulnerabilidad detectada.

10. Multi-Tenencia (Arquitectura SaaS)

Castilfac es un SaaS multi-inquilino donde cada empresa cliente tiene sus datos aislados logicamente dentro de la misma base de datos. La estrategia de aislamiento se basa en el campo `empresa_id` presente en la mayoria de tablas.

Estrategia de Aislamiento

- Base de datos unica con logica por `empresa_id`
- El middleware de tenant (`utils/tenant.js`) extrae el contexto del JWT
- Todas las queries se filtran automaticamente por `empresa_id`
- Cada empresa tiene su propio catalogo, precios, clientes y configuracion
- Superadmin puede acceder a todas las empresas

Modelo de Suscripcion

Cada empresa tiene un plan asociado (gratis, basico, premium) gestionado mediante Stripe. El middleware `checkSuscripcion.middleware.js` verifica que la empresa tenga una suscripcion activa antes de permitir operaciones restringidas. El estado de suscripcion se almacena en la tabla `empresas` y se sincroniza via webhooks de Stripe.

Roles dentro de cada Empresa

Cada empresa tiene su propia jerarquia de usuarios:

- admin: administrador de la empresa (gestion completa)
- operario: acceso limitado a pedidos y tareas
- superadmin: rol global fuera de la empresa (gestion del SaaS)

11. Stripe y Suscripciones

El sistema de pagos esta integrado con Stripe para gestionar suscripciones periodicas (mensuales/anuales) de las empresas clientes del SaaS.

Flujo de Pago

- 1. El usuario administrador accede a la pagina de planes y selecciona un plan
- 2. El backend crea una sesion de checkout en Stripe y devuelve la URL
- 3. El usuario es redirigido al portal de pago de Stripe
- 4. Stripe procesa el pago y envia un webhook al backend
- 5. El webhook actualiza el estado de suscripcion de la empresa
- 6. Stripe envia webhooks periodicos para renovaciones, fallos y cancelaciones

Webhooks

El endpoint `/api/stripe/webhook` recibe eventos de Stripe con el body en crudo (antes de `express.json()`) para verificar la firma criptografica. Eventos gestionados: `checkout.session.completed`, `invoice.paid`, `invoice.payment_failed`, `customer.subscription.updated`, `customer.subscription.deleted`.

Portal de Cliente

Stripe Customer Portal permite a los administradores gestionar su suscripcion: cambiar de plan, actualizar metodo de pago, descargar facturas, cancelar la suscripcion. Se accede desde el panel de administracion.

12. Internacionalizacion (i18n)

La aplicacion soporta espanol (es) e ingles (en) mediante la libreria @ngx-translate/core. Los archivos de traduccion se almacenan en public/i18n/ y se cargan via HttpLoader.

Detalles de Implementacion

- es.json: ~50KB con todas las cadenas en espanol (idioma principal)
- en.json: todas las cadenas traducidas al ingles
- Servicio LanguageService: gestiona el idioma activo y lo persiste
- Deteccion automatica del idioma del navegador
- Selector de idioma en la interfaz de usuario
- Cobertura completa de la aplicacion: login, registro, paneles, etc.

13. Docker y Despliegue

El despliegue se realiza mediante Docker Compose con tres servicios orquestados. Se gestiona a traves de Portainer CE, expuesto publicamente via Cloudflare Tunnel.

Servicios Docker

castilfac_db	MariaDB 10.11	Base de datos persistente
castilfac_api	Node.js	API Express (puerto 3000)
castilfac_web	Nginx	Frontend Angular (puerto 8080)

Docker Compose

El docker-compose.yml define los tres servicios con variables de entorno inyectadas desde Portainer. La base de datos usa un volumen persistente (castilfac_data) montado en /var/lib/mysql para los datos.

Nginx (Frontend)

nginx.conf configura el servidor web para servir el build de Angular, redirigir las peticiones /api/ al backend, y manejar el SSR de Angular Universal cuando esta activo.

Despliegue en Produccion

- Servidor: Ubuntu Server con Docker Engine
- Gestion: Portainer CE con stacks Docker Compose
- Acceso publico: Cloudflare Tunnel -> castilfac.pablosrv.com
- Variables de entorno: gestionadas en Portainer (no en .env)
- Puertos: backend 3000 interno, frontend 8080 -> 80
- Base de datos: MariaDB con volumen persistente

14. Estructura del Repositorio

```
castilfac/
|-- README.md, .gitignore, .env.example
|-- docker-compose.yml          # Orquestacion Docker
|-- auditoria.md                # Auditoria de seguridad
|-- solucion-auditoria.md       # Soluciones aplicadas
|-- backend/                    # Node.js + Express
|   |-- Dockerfile
|   |-- package.json
|   |-- src/
|       |-- app.js              # Entry point
|       |-- config.js           # Variables de entorno
|       |-- mailer.js           # SMTP
|       |-- data/db.js          # Sequelize connection
|       |-- controllers/        # 18 controladores
|       |-- models/             # 14 modelos Sequelize
|       |-- routes/             # 16 archivos de rutas
|       |-- middlewares/        # auth, tenant, rate-limit
|       |-- utils/              # seguridad, regex, logger
|       |-- validators/         # Validacion de entrada
|-- frontend/                   # Angular 21
    |-- Dockerfile, nginx.conf
    |-- package.json
    |-- angular.json, tailwind.config.js
    |-- public/
    |   |-- i18n/es.json, en.json
    |   |-- img/
    |-- src/
        |-- index.html, main.ts, server.ts
        |-- app/
            |-- components/      # 40+ componentes
            |-- guards/          # 3 guards
            |-- interceptors/    # Auth HTTP interceptor
            |-- interfaces/      # 14 TS interfaces
            |-- services/        # 19 servicios HTTP
```

15. Ejecucion Local

Requisitos

- Node.js 22+ con npm
- MariaDB 10.11 o MySQL 8+
- Docker + Docker Compose (opcional, para produccion)
- Cuenta Stripe para pruebas (modo test)
- Cliente ID de Google OAuth (opcional)

1. Clonar y configurar

```
git clone https://github.com/Pablocstlpz/castilfac.git
cd castilfac/backend
cp .env.example .env # Editar con tus datos
```

2. Backend

```
cd backend
npm install
npm run dev # http://localhost:3000
```

3. Frontend

```
cd frontend
npm install
ng serve # http://localhost:4200
```

4. Con Docker (produccion)

```
cd castilfac
docker compose up -d
# Frontend: http://localhost:8080
# Backend: http://localhost:3000
```

5. Variables de Entorno Criticas

Configurar en .env (desarrollo) o Portainer (produccion):

- DB_HOST, DB_USER, DB_PASSWORD, DB_DATABASE (MariaDB)
- SECRET_KEY y REFRESH_SECRET_KEY (claves JWT)
- STRIPE_SECRET_KEY y STRIPE_WEBHOOK_SECRET
- FRONTEND_URL y URL (dominio publico)
- GOOGLE_CLIENT_ID (opcional, para login con Google)

16. Mejoras Potenciales

- Migraciones de base de datos con Sequelize CLI en lugar de sync()
- Pruebas automatizadas (unitarias e integracion)
- CI/CD pipeline con GitHub Actions
- Monitorizacion y alertas (logs centralizados con Winston/Sentry)
- Cache con Redis para mejorar rendimiento
- WebSockets para notificaciones en tiempo real
- Panel de metricas y analiticas de uso
- API publica documentada con OpenAPI/Swagger
- Sistema de backups automaticos de base de datos
- Modo offline / PWA para operarios en taller
- Integracion con servicios de mensajeria (WhatsApp/Telegram)
- Facturacion electronica integrada
- Notificaciones push para cambios de estado en pedidos
- Version multi-idioma completa (mas idiomas)
- Dashboard de KPIs para direccion de la empresa